

This problem requires finding the second shortest path from node 1 to node N in an undirected graph where backtracking and revisiting nodes/edges are explicitly allowed, meaning the solution cannot simply find the second simplest path but must consider paths that may include cycles or repeated segments that make the total distance longer than the optimal path yet minimal among all such longer alternatives. The key insight is that the second best path can be found by maintaining both the shortest and second shortest distances to each node during a modified Dijkstra's algorithm traversal, where we relax edges not only from the shortest known path but also from the second shortest, ensuring we discover paths that might deviate slightly from the optimal route through small detours or alternative segments while remaining efficient with $O((V+E) \log V)$ complexity. This approach systematically explores all possibilities where the path length exceeds the shortest path but is minimized, correctly handling cases where the second shortest path might involve backtracking or reusing edges rather than being a completely distinct simple path from source to destination.

Steps:

1. Initialize Arrays

- Create two arrays: dist1 for shortest distances and dist2 for second shortest distances
- Set all values in both arrays to a very large number (infinity)
- Create a min-priority queue that will store pairs of (distance, node)

2. Setup Starting Node

- Set the shortest distance to node 1 as 0
- Push (0, 1) into the priority queue

3. Process the Queue

- While the priority queue is not empty:
 - Remove the smallest (distance, node) pair from the queue
 - If this distance is larger than the current second shortest for that node, skip it
 - Otherwise, examine all neighbors connected to this node

4. Check Each Neighbor

- For each neighbor connected by an edge with weight w:
 - Calculate the new distance = current distance + w
 - If this new distance is better than the current shortest distance to the neighbor:
 - Update the second shortest to be the old shortest value
 - Update the shortest to be the new distance
 - Push the new distance and neighbor into the queue

- Else if the new distance is between the shortest and second shortest:
 - Update the second shortest to be the new distance
 - Push the new distance and neighbor into the queue

5. **Get the Answer**

- After processing all nodes, the second shortest distance to node N is stored in `dist2[N]`
- This value is the length of the second best path from node 1 to node N

Input:

2

3 3

1 2 100

2 3 200

1 3 50

4 4

1 2 100

2 4 200

2 3 250

3 4 100

Execution:

1. **Initialize:**

- `dist1 = [0, ∞, ∞]`, `dist2 = [∞, ∞, ∞]`
- Queue: `[(0,1)]`

2. **Process (0,1):**

- `1→2: new=100 < ∞ → dist1[2]=100, push (100,2)`
- `1→3: new=50 < ∞ → dist1[3]=50, push (50,3)`

- Queue: [(50,3), (100,2)]
- 3. **Process (50,3):**
 - $3 \rightarrow 1$: new=100 > 0 but < $\infty \rightarrow \text{dist2}[1]=100$, push (100,1)
 - $3 \rightarrow 2$: new=250 > 100 but < $\infty \rightarrow \text{dist2}[2]=250$, push (250,2)
 - Queue: [(100,2), (100,1), (250,2)]
- 4. **Process (100,2):**
 - $2 \rightarrow 1$: new=200 > 100 ($\text{dist2}[1]$) $\rightarrow$ skip
 - $2 \rightarrow 3$: new=300 > 50 but < $\infty \rightarrow \text{dist2}[3]=300$, push (300,3)
 - Queue: [(100,1), (250,2), (300,3)]
- 5. **Process (100,1):**
 - $1 \rightarrow 2$: new=200 > 100 but < 250 $\rightarrow \text{dist2}[2]=200$, push (200,2)
 - $1 \rightarrow 3$: new=150 > 50 but < 300 $\rightarrow \text{dist2}[3]=150$, push (150,3)
 - Queue: [(150,3), (200,2), (250,2), (300,3)]
- 6. **Process (150,3):**
 - Target reached with second shortest = 150
 - **Answer: 150**

Output:

150

Pseudocode:

FOR each test case:

 READ graph

 INIT dist1=INF, dist2=INF

 dist1[1] = 0

 PUSH (0,1) to min-heap

 WHILE heap not empty:

 POP (d,u)

IF $d > \text{dist2}[u]$: SKIP

FOR each neighbor v with weight w :

$\text{new_dist} = d + w$

 IF $\text{new_dist} < \text{dist1}[v]$:

$\text{dist2}[v] = \text{dist1}[v]$

$\text{dist1}[v] = \text{new_dist}$

 PUSH (new_dist , v)

 ELSE IF new_dist between $\text{dist1}[v]$ and $\text{dist2}[v]$:

$\text{dist2}[v] = \text{new_dist}$

 PUSH (new_dist , v)

OUTPUT $\text{dist2}[N]$